



REFERÊNCIA DE API PARA O DESENVOLVEDOR

Biblioteca libK044AVT.so

— Rotinas da API —

Teclado industrial PS/2 TEC44AVT — funções, parâmetros e códigos de retorno de cada rotina disponível ao desenvolvedor.

Produto TEC44AVT

Público-alvo Desenvolvedor

Versão 1.0 – Setembro de 2026

Índice

1. CICLO DE VIDA	4
2. INFORMACOES DO DISPOSITIVO E DA BIBLIOTECA	6
3. DISPLAY — ALTO NIVEL	7
4. CONTROLE DE CURSOR	9
5. COMANDOS PS/2 PADRAO	11
6. Avanttec TEC44FST Keyboard-Mouse	12
7. MONTAGEM DE DISPOSITIVOS E LEITURA DE EVENTOS	14
8. EVENTOS — Leitura Assincrona e Sincrona (event loop)	16
9. KEYECHO — EXIBICAO DE TECLAS NO DISPLAY	18
10. MODULO DE IMPRESSAO DIGITAL (fingerprint)	20

Códigos de retorno

Todas as rotinas retornam um int com um dos códigos abaixo, salvo indicação em contrário. K044_OK (0) sempre indica sucesso.

K044_OK	0	sucesso
K044_ERR_PERM	-1	permissao negada (requer root ou setcap CAP_SYS_RAWIO)
K044_ERR_TIMEOUT	-2	dispositivo nao respondeu no prazo
K044_ERR_NODEV	-3	dispositivo nao encontrado / ID invalido
K044_ERR_RANGE	-4	parametro fora do intervalo valido
K044_ERR_INVALID	-5	parametro invalido (NULL, etc.)
K044_ERR_BUSY	-6	recurso ocupado (ex.: event loop ja ativo)
K044_ERR_RESEND	-7	dispositivo solicitou retransmissao
K044_ERR_FP	-8	erro do modulo fingerprint
K044_ERR_FP_TIMEOUT	-9	timeout aguardando o modulo fingerprint

1. CICLO DE VIDA

k044_open(void)

Abre o dispositivo com configuracao padrao.

Retorno: K044_OK ou erro

k044_open_ex(const k044_config_t *cfg)

Abre o dispositivo com configuracao personalizada. cfg=NULL usa padrao.

Retorno: K044_OK ou erro

k044_close(void)

Fecha o dispositivo.

Retorno: void

k044_lock(void)

Captura o mutex do barramento PS/2.

Retorno: K044_OK

k044_unlock(void)

Libera o mutex capturado por k044_lock().

Retorno: void

k044_open() vs k044_open_ex()

k044_open() abre o dispositivo com a configuração padrão (portas PS/2 0x60/0x64, timeouts padrão); k044_open_ex() abre com uma k044_config_t personalizada, útil quando você precisa mudar portas, timeouts ou nível de log. Chame k044_config_init() para preencher os padrões e só altere o campo que precisar; k044_open_ex(NULL) equivale a k044_open().

Para que servem k044_lock / k044_unlock

Garantem atomicidade quando você precisa fazer várias chamadas seguidas sem que outra thread (ex.: a do event loop) intercale uma transação PS/2 no meio. A maioria dos programas nunca precisa chamá-los diretamente, pois as próprias rotinas da biblioteca já usam o lock internamente quando fazem uma transação composta.

Estrutura k044_config_t

```
typedef struct {
    uint16_t port_data;           // porta de dados PS/2    (padrão: 0x60)
    uint16_t port_status;        // porta de status PS/2   (padrão: 0x64)
    uint32_t write_timeout_ms;    // timeout de escrita, ms (padrão: 100)
    uint32_t read_timeout_ms;     // timeout de leitura, ms (padrão: 200)
    uint32_t reset_wait_ms;       // espera após reset, ms  (padrão: 1000)
    int      max_retries;         // tentativas em RESEND   (padrão: 3)
    int      log_level;          // nível de log            (padrão: NONE)
} k044_config_t;
```

CAMPO	PADRÃO	DESCRIÇÃO
port_data	0x60	Porta de dados do barramento PS/2
port_status	0x64	Porta de status do barramento PS/2
write_timeout_ms	100 ms	Espera por confirmação ao enviar um comando
read_timeout_ms	200 ms	Espera por uma resposta/byte do dispositivo
reset_wait_ms	1000 ms	Espera após o comando de reset (0xFF)
max_retries	3	Retransmissões automáticas em K044_ERR_RESEND
log_level	LOG_NONE	Verbosidade dos logs internos (NONE/ERROR/WARN/DEBUG/TRACE)

Na prática

Os campos mais ajustados são read_timeout_ms/write_timeout_ms (hardware mais lento que o normal, ex.: junto com o sensor de digital) e log_level (para depurar problemas de comunicação); os demais quase nunca precisam mudar do padrão.

2. INFORMACOES DO DISPOSITIVO E DA BIBLIOTECA

k044_version(void)

Retorna a string de versao da biblioteca (ex.: "1.0.3").

*Retorno: const char**

k044_read_id(uint8_t id[2])

Le o identificador do dispositivo (comando 0xF2). Sucesso: id={0xAB,0x83}.

Retorno: K044_OK ou erro

k044_firmware_version(char *buf, size_t len)

Le a versao do firmware como string (comando 0xF1), ex.: "K044P001A".

len minimo recomendado: 16.

Retorno: K044_OK ou erro

k044_read_fw_ver(uint8_t *major, uint8_t *minor, uint8_t *patch)

Le a versao numerica do firmware (comando 0xAF). Os ponteiros NULL.

Retorno: K044_OK, ou K044_ERR_NODEV se o firmware nao suportar comando 0xAF

k044_fw_ver_str(char *buf, size_t len)

Formata "major.minor.patch" a partir do ultimo k044_read_fw_ver() bem sucedido. len minimo: 12.

Retorno: K044_OK ou erro

k044_read_serial(char *buf, size_t len)

Le o numero de serie do equipamento, 7 chars ASCII.

len minimo: 8.

Retorno: K044_OK, ou K044_ERR_NODEV se o firmware nao suportar o comando

3. DISPLAY — ALTO NIVEL

k044_write_cgram_preset(uint8_t pattern, uint8_t slot)

Grava um padrao pre-definido (K044_CGRAM_PRESET_*) em um slot da CGRAM (0-7)

Retorno: K044_OK ou erro

k044_write_char(uint8_t c)

Escreve 1 byte na posicao atual do cursor. Bytes < 0x20 sao interpretados como comandos de controle (K044_BS, K044_CR, etc.).

Retorno: K044_OK ou erro

k044_write_char_raw(uint8_t c)

Escreve 1 byte direto na DDRAM, sem interpretar codigos de controle — usar para exibir caracteres CGRAM (codigos 0-7).

Retorno: K044_OK ou erro

k044_write_string(const char *s)

Escreve string terminada em '\0' (max 80 chars, duas linhas).

Retorno: K044_OK ou erro

k044_write_buf(const uint8_t *buf, size_t len)

Escreve um buffer de bytes no display.

Retorno: K044_OK ou erro

k044_write_line(uint8_t row, const char *text)

Escreve uma linha inteira (0 ou 1), completando com espacos ate a col 39.

Retorno: K044_OK ou erro

k044_display_shift(int direction)

Desloca o display inteiro (instrucao nativa do controlador HD44780)

direction: [0] = esquerda, [!= 0] = direita.

Retorno: K044_OK ou erro

k044_write_display(const char *line1, const char *line2)

Limpa o display e escreve as duas linhas de uma vez. NULL = linha vazia.

Retorno: K044_OK ou erro

k044_write_pos(uint8_t row, uint8_t col, const char *fmt, ...)

Escreve string formatada (estilo printf) na posicao (row, col).

row: 0-1 | col: 0-39 | fmt/...: como printf.

Retorno: K044_OK ou erro

k044_scroll_line(uint8_t row, const char *text, unsigned int delay_ms)

Rola texto horizontalmente numa linha, um loop, janela deslizando.

Retorno: K044_OK ou erro

k044_scroll_start(uint8_t row, uint8_t col_start, uint8_t width,

const char *text, unsigned int delay_ms, int repeat)

Inicia scroll continuo em uma janela (col_start, width) da linha.

repeat: 0 = loop infinito, N = numero de loops.

Retorno: K044_OK ou erro

k044_scroll_stop(void)

Para o scroll continuo iniciado por k044_scroll_start().

Retorno: K044_OK ou erro

4. CONTROLE DE CURSOR

k044_clear(void)

Limpa o display inteiro.

Retorno: K044_OK/erro

k044_wait_busy(void)

Aguarda o dispositivo ficar livre (polling ECHO).

Retorno: K044_OK, ou K044_ERR_TIMEOUT

k044_home(void)

Cursor para (0,0).

Retorno: K044_OK/erro

k044_set_cursor(uint8_t row, uint8_t col)

Posiciona cursor (0-based). row: 0-1 | col: 0-39.

Retorno: K044_OK, ou K044_ERR_RANGE se fora da faixa

k044_cursor_on(void)

Ativa exibicao do cursor.

Retorno: K044_OK/erro

k044_cursor_off(void)

Oculto o cursor.

Retorno: K044_OK/erro

k044_carriage_return(void)

Cursor para coluna 0 da linha atual.

Retorno: K044_OK/erro

k044_line_feed(void)

Cursor para a linha 2.

Retorno: K044_OK/erro

k044_cursor_up(void)

Cursor para a linha 1.

Retorno: K044_OK/erro

k044_backspace(void)

Cursor uma posicao a esquerda (apaga).

Retorno: K044_OK/erro

k044_tab(void)

Avanca cursor uma posicao.

Retorno: K044_OK/erro

k044_cursor_inc(void)

Avanca cursor sem apagar (= tab).

Retorno: K044_OK/erro

k044_cursor_dec(void)

Recua cursor sem apagar.

Retorno: K044_OK/erro

k044_erase_eol(void)

Apaga da posicao atual ate fim da linha.

Retorno: K044_OK/erro

k044_bell(void)

Sinal sonoro (sem efeito neste hardware).

Retorno: K044_OK/erro

5. COMANDOS PS/2 PADRAO

k044_reset(void)

Reinicia o firmware do dispositivo (comando 0xFF).

Retorno: K044_OK ou erro

k044_set_leds(uint8_t leds)

Controla os LEDs do teclado (comando 0xED). leds: bitmask de K044_LED_SCROLLLOCK | K044_LED_NUMLOCK | K044_LED_CAPSLOCK.

Retorno: K044_OK ou erro

k044_set_typematic(uint8_t rate)

Define a taxa de repetição de teclas (comando 0xF3, encoding PS/2 padrão).

Retorno: K044_OK ou erro

6. Avanttec TEC44FST Keyboard-Mouse

k044_uinput_enable(void)

Cria um teclado virtual (/dev/uinput).

Retorno: K044_OK, K044_ERR_NODEV (uinput inacessível) ou K044_ERR_BUSY

k044_uinput_disable(void)

Desativa o teclado virtual.

Retorno: K044_OK ou erro

k044_uinput_is_active(void)

Verifica se o teclado virtual (uinput) está ativo no momento.

Retorno: 1 se ativo, 0 caso contrário

k044_mouse_enable(void)

Cria um mouse virtual (/dev/uinput) para repassar ao sistema os eventos de movimento/clique do dispositivo.

Retorno: K044_OK, K044_ERR_NODEV ou K044_ERR_BUSY

k044_mouse_disable(void)

Desativa o mouse virtual.

Retorno: K044_OK ou erro

k044_mouse_is_active(void)

Verifica se o mouse virtual está ativo no momento.

Retorno: 1 se ativo, 0 caso contrário

k044_set_mouse_auto(int enable)

Se enable=1 (chamar ANTES de k044_open()), ativa mouse+event_loop automaticamente ao final de k044_open().

Retorno: void

k044_is_device_dead(void)

Verifica se o dispositivo foi marcado inoperante após falha irreversível de protocolo. Quando 1, toda E/S retorna K044_ERR_NODEV imediatamente.

Retorno: 0 = operacional, 1 = inoperante

k044_aux_enable(void)

Habilita o teclado auxiliar (PS/2, 110 teclas) conectado ao dispositivo.

aTENCAO: So utilize a rotina com o teclado auxiliar fisicamente conectado

— o sistema pode travar aguardando resposta.

Retorno: K044_OK ou erro

k044_aux_disable(void)

Desabilita o teclado auxiliar.

Retorno: K044_OK ou erro

7. MONTAGEM DE DISPOSITIVOS E LEITURA DE EVENTOS

Esta seção orienta, de forma direta, como combinar as rotinas da biblioteca de acordo com a arquitetura e os requisitos do seu aplicativo. Abaixo são apresentados a ordem correta das chamadas, os cenários típicos de uso e a escolha do modo de leitura ideal.

Cenários Típicos de Integração

1. Apenas Display/Cursor (Teclado inativo no sistema)

Objetivo:

Utilizar apenas a interface visual, sem que o teclado envie comandos para o sistema operacional.

Fluxo de Chamadas:

k044_open() seguido estritamente pelas rotinas de controle de display/cursor. Não inicialize uinput, mouse ou dispositivos auxiliares (aux).

2. Terminal Interativo / Menu (Teclado ativo e funcional)

Objetivo:

Permitir que o usuário digite e navegue normalmente pelo sistema enquanto o software roda em segundo plano.

Fluxo de Chamadas:

Logo após o k044_open(), chame a sequência: k044_uinput_enable() → k044_mouse_enable() → k044_start_event_loop() → k044_aux_enable() (caso haja teclado auxiliar físico conectado).

3. Teclado Bloqueado (Aguardando validação/condição)

Objetivo:

Reter a digitação (ex.: travar o teclado até que uma senha seja validada ou uma digital seja reconhecida).

Fluxo de Chamadas:

Execute apenas k044_open(). As teclas serão capturadas internamente pela biblioteca, mas não serão injetadas no sistema operacional. Após a validação da condição, libere o fluxo invocando as funções de inicialização (uinput, mouse, aux e event_loop).

4. Eco de Teclas no Display (Campos de texto / PIN)

Objetivo:

Mostrar os caracteres digitados diretamente na tela (ou asteriscos para senhas) de forma otimizada.

Fluxo de Chamadas:

Invoque diretamente k044_keyecho_start(). Esta função inicializa o laço de eventos (event loop) de forma autônoma se ele ainda não estiver em execução, dispensando a chamada manual de k044_start_event_loop().

5. Ativação Automática do Mouse PS/2

Objetivo:

Garantir o funcionamento imediato e automático do mouse desde a inicialização do software.

Fluxo de Chamadas:

Chame obrigatoriamente `k044_set_mouse_auto(1)` antes de executar o `k044_open()`. Esse procedimento dispensa a ativação manual posterior via `k044_mouse_enable()` ou `k044_start_event_loop()`.

Regra Geral sobre o Event Loop

A execução do `k044_start_event_loop()` é obrigatória para habilitar os seguintes recursos: tradução e injeção de teclas via subsistema uinput; recebimento de dados por leitura síncrona (`k044_read_event()`) ou assíncrona (`k044_set_event_callback()`); e o funcionamento da rotina de eco em tela (`keyecho`).

Atenção

As funções `k044_read_event()` e `k044_keyecho_start()` gerenciam e iniciam o event loop sozinhas se necessário. Contudo, a função `k044_uinput_enable()` NÃO o inicia. Se você habilitar o uinput sem disparar o event loop em algum momento, nenhuma tecla será repassada ao sistema operacional.

Modos de Leitura de Evento — Qual Usar?

MODO	FUNÇÃO BASE	QUANDO UTILIZAR
Callback (Assíncrono)	<code>k044_set_event_callback</code>	Aplicações com interface gráfica (GUI) ou que possuam um laço de repetição próprio que não pode ser bloqueado aguardando a interação do usuário.
Síncrono (Bloqueante)	<code>k044_read_event</code>	Programas de terminal baseados em fluxos sequenciais simples (onde o sistema necessariamente espera uma tecla, processa o dado e aguarda a próxima ação).
Keyecho (Automatizado)	<code>k044_keyecho_start</code>	Quando o único objetivo é imprimir o que foi digitado no display. A biblioteca gerencia a escrita de forma autônoma e rápida, eliminando a necessidade de criar códigos manuais de tratamento de eventos.

Atenção ao Desempenho (Gargalo do Barramento)

Evite chamar rotinas manuais de display (como `k044_write_pos`, etc.) dentro de um loop de captura de eventos gerenciado por `k044_read_event()` ou pela função de callback. Como as operações de escrita no visor e o escaneamento do teclado compartilham o mesmo barramento físico PS/2, cada transação de tela concorrente causará atrasos perceptíveis (lag) na digitação do usuário. Quando precisar exibir caracteres digitados no display, use sempre a função nativa `k044_keyecho_start()`, que implementa uma via de escrita otimizada para mitigar esse problema.

8. EVENTOS — Leitura Assíncrona e Síncrona (event loop)

O monitoramento de eventos permite que o programa seja notificado sobre ações dos dispositivos sem a necessidade de consulta constante (polling). Existem dois modelos de implementação, escolha conforme a arquitetura do seu aplicativo:

1. Modelo Assíncrono via Callback

Funções:

`k044_set_event_callback + k044_start_event_loop`

Funcionamento:

O evento é disparado automaticamente em uma thread interna dedicada sempre que uma nova ação ocorre.

Quando usar:

Ideal para aplicações com interface gráfica (GUI) ou que possuam um loop principal próprio que não pode ser bloqueado.

Exemplo:

Um sistema de PDV (Frente de Caixa) que precisa continuar processando comandos na tela enquanto aguarda o cliente passar o cartão.

Atenção:

A função de callback deve ser leve, rápida e thread-safe (evite operações pesadas de I/O ou bloqueios).

2. Modelo de Leitura Síncrona (Bloqueante)

Função:

`k044_read_event`

Funcionamento:

O programa executa um loop simples que bloqueia a execução até que o próximo evento ocorra ou o tempo limite (timeout) seja atingido.

Quando usar:

Recomendado para aplicações de terminal (CLI) com fluxo sequencial, no estilo "espera o evento — processa — espera o próximo". É um modelo mais simples e linear de codificar do que o callback.

NOTA DE ENCERRAMENTO

Sempre chame a função `k044_stop_event_loop()` quando não for mais necessário monitorar os periféricos, para liberar os recursos do sistema.

Tipos de Eventos Disponíveis (k044_event_type_t)

CATEGORIA	EVENTO(S)	DESCRIÇÃO
Teclado	K044_EVT_KEY_MAKE K044_EVT_KEY_BREAK	Tecla pressionada / tecla solta
PIN	K044_EVT_PIN_KEY	Dígito do PIN inserido
Cartão	K044_EVT_CARD_START K044_EVT_CARD_DATA K044_EVT_CARD_END K044_EVT_CARD_ERROR	Eventos da leitura de cartão magnético (início, dado, fim, erro)
Dados Puros	K044_EVT_RAW	Byte bruto do barramento PS/2

9. KEYECHO — EXIBICAO DE TECLAS NO DISPLAY

k044_keyecho_cfg_init(k044_keyecho_cfg_t *cfg)

Preenche cfg com valores padrao (row=0, max_chars=32, confirm=Enter, etc.).

Retorno: void

k044_keyecho_start(const k044_keyecho_cfg_t *cfg, k044_keyecho_cb_t cb, void *userdata)

Inicia a exibicao de teclas pressionadas no display (so teclado principal).

cfg=NULL usa padrao. cb=NULL = sem notificacao.

Retorno: K044_OK, ou K044_ERR_RANGE se a config for invalida

k044_keyecho_stop(void)

Para o keyecho e apaga a area usada no display.

Retorno: K044_OK ou erro

k044_keyecho_get_buffer(char *buf, uint8_t *len)

Copia o conteudo atual do buffer para buf (min. 16 bytes).

Retorno: K044_OK, ou K044_ERR_INVALID se buf/len forem NULL

k044_keyecho_get_cursor(uint8_t *pos)

Retorna a posicao do cursor dentro do buffer (0-based).

Retorno: K044_OK, ou K044_ERR_INVALID se pos for NULL

k044_keyecho_clear(void)

Limpa o buffer programaticamente (equivale a pressionar clear_scancode).

Retorno: K044_OK ou erro

Estrutura k044_keyecho_cfg_t

```
typedef struct {
    uint8_t row;           // linha do display: 0 ou 1 (padrão: 0)
    uint8_t col_start;     // coluna inicial: 0-14 (padrão: 0)
    uint8_t max_chars;     // máx. chars visíveis: 1-40 (padrão: 32)
    uint16_t clear_scancode; // tecla que limpa o buffer (0=desativado)
    uint16_t confirm_scancode; // tecla que confirma entrada (padrão: 0x5A)
    int pin_mode;          // 1 = mostra '*' no lugar (padrão: 0)
    int echo_enabled;      // 1 = escreve no display (padrão: 1)
} k044_keyecho_cfg_t;
```

CAMPO	PADRÃO	DESCRIÇÃO
row / col_start	0 / 0	Onde no display o texto aparece
max_chars	32	Até onde o texto pode crescer ($\text{col_start} + \text{max_chars} \leq 40$)
clear_scancode	0	Tecla que limpa o buffer inteiro (0 = desativado)
confirm_scancode	0x5A	Tecla que confirma a entrada (dispara CONFIRMED)
pin_mode	0	1 = mostra '*' em vez do caractere real
echo_enabled	1	0 = mantém buffer/callback sem escrever no display

Eventos k044_keyecho_event_t

São os motivos pelos quais o callback passado a `k044_keyecho_start()` é chamado, cada vez que algo muda no buffer de digitação.

EVENTO	DESCRIÇÃO
K044_KEYECHO_CHAR	Uma tecla normal foi digitada e adicionada ao buffer
K044_KEYECHO_CLEARED	A tecla de limpeza (<code>clear_scancode</code>) foi pressionada
K044_KEYECHO_BACKSPACE	Um caractere foi apagado à esquerda do cursor
K044_KEYECHO_DELETE	Um caractere foi apagado na posição atual do cursor
K044_KEYECHO_CONFIRMED	Tecla de confirmação pressionada — leia o buffer aqui
K044_KEYECHO_FULL	O buffer atingiu <code>max_chars</code> e a tecla foi ignorada

Na prática

A maioria dos programas só precisa tratar `K044_KEYECHO_CONFIRMED` (para pegar o resultado final) e talvez `K044_KEYECHO_FULL` (para dar feedback de limite excedido); os demais eventos existem para casos mais avançados de sincronização de estado.

10. MODULO DE IMPRESSAO DIGITAL (fingerprint)

k044_fp_init(void)

Inicializa a comunicacao com o modulo fingerprint. Nao requer configuracao adicional; chame uma vez apos k044_open(), antes de qualquer outra rotina k044_fp_*.

Retorno: K044_OK ou erro

k044_fp_get_image(void)

Captura a imagem da digital apoiada sobre o sensor no momento da chamada. E' sempre o primeiro passo de um cadastro/identificacao manual, chamada antes de k044_fp_gen_char().

Retorno: K044_OK, ou K044_ERR_FP se o sensor estiver vazio (nenhum dedo) ou a leitura falhar

k044_fp_gen_char(uint8_t buffer_id)

Gera o template (assinatura da digital) a partir da ultima imagem capturada por k044_fp_get_image(), e guarda no CharBuffer indicado. buffer_id: qual CharBuffer (memoria temporaria do sensor) recebe o template gerado — 1 ou 2. Um cadastro manual usa os dois buffers (uma leitura em cada) antes de mescla-los com k044_fp_reg_model().

Retorno: K044_OK ou erro

k044_fp_match(void)

Compara os templates atualmente guardados nos CharBuffers 1 e 2 entre si (nao contra o banco de dados) e informa se pertencem ao mesmo dedo.

Retorno: K044_OK se os templates coincidem, K044_ERR_FP se nao

k044_fp_search(uint8_t buffer_id, uint16_t start_page, uint16_t page_num, uint16_t *found_id)

Busca o template de um CharBuffer no banco de dados do modulo.

buffer_id: qual CharBuffer contem o template a procurar — 1 ou 2.

start_page: posicao inicial do banco a partir de onde a busca comeca (normalmente 0).

page_num: quantas posicoes, a partir de start_page, devem ser varridas na busca (ex.: 1000 para cobrir todo o banco).

found_id: ponteiro de saida — recebe o ID/posicao do template encontrado quando a busca tem sucesso.

Retorno: K044_OK se encontrado, K044_ERR_FP se nao

k044_fp_reg_model(void)

Mescla os templates guardados nos CharBuffers 1 e 2 em um unico modelo, pronto para ser gravado no banco com k044_fp_store(). Passo intermediario de um cadastro manual (entre gerar os templates das leituras e armazená-los).

Retorno: K044_OK ou erro

k044_fp_store(uint8_t buffer_id, uint16_t page_id)

Grava o modelo atualmente no CharBuffer indicado no banco de dados do modulo, numa posicao especifica.

buffer_id: qual CharBuffer contem o modelo a gravar — 1 ou 2 (depois de chamar k044_fp_reg_model(), o modelo mesclado fica disponivel em ambos).

page_id: posicao/ID no banco onde gravar (0-999) — e' o identificador que sera devolvido depois por k044_fp_search()/k044_fp_auto_identify().

Retorno: K044_OK ou erro

k044_fp_load_char(uint8_t buffer_id, uint16_t page_id)

Carrega um modelo ja gravado no banco de volta para um CharBuffer, para poder compara-lo (k044_fp_match()) com uma nova leitura.

buffer_id: para qual CharBuffer carregar o modelo — 1 ou 2.

page_id: de qual posicao do banco (0-999) carregar.

Retorno: K044_OK ou erro

k044_fp_delete_char(uint16_t page_id, uint16_t count)

Remove um ou mais templates do banco de dados.

page_id: posicao inicial (0-999) a partir de onde remover.

count: quantos registros consecutivos remover a partir de page_id (1 para apagar so aquela posicao).

Retorno: K044_OK ou erro

k044_fp_empty(void)

Limpa todo o banco de dados de templates do modulo — remove todos os cadastros de uma vez. Nao pode ser desfeito.

Retorno: K044_OK ou erro

k044_fp_valid_template_count(uint16_t *count)

Conta quantos templates validos existem no banco de dados no momento.

count: ponteiro de saida — recebe o numero de templates encontrados.

Retorno: K044_OK ou erro

k044_fp_led_off_fast(void)

Ajustam o LED do sensor para um estado fixo (cor azul, ciclos=0) usando um comando pre-gravado no firmware, sem parametros — mais rapido e mais confiavel na transmissao do que k044_fp_led_config(), mas sempre na cor azul e sem possibilidade de transicao de cor. Use k044_fp_led_config() quando precisar de outra cor ou de uma transicao entre cores.

Retorno: K044_OK ou erro (cada uma)

k044_fp_led_on_fast(void)

Ajustam o LED do sensor para um estado fixo (cor azul, ciclos=0) usando um comando pre-gravado no firmware, sem parametros — mais rapido e mais confiavel na transmissao do que k044_fp_led_config(), mas sempre na cor azul e sem possibilidade de transicao de cor. Use k044_fp_led_config() quando precisar de outra cor ou de uma transicao entre cores.

Retorno: K044_OK ou erro (cada uma)

k044_fp_led_flashing_fast(void)

Ajustam o LED do sensor para um estado fixo (cor azul, ciclos=0) usando um comando pre-gravado no firmware, sem parametros — mais rapido e mais confiavel na transmissao do que k044_fp_led_config(), mas sempre na cor azul e sem possibilidade de transicao de cor. Use k044_fp_led_config() quando precisar de outra cor ou de uma transicao entre cores.

Retorno: K044_OK ou erro (cada uma)

k044_fp_led_breathing_fast(void)

Ajustam o LED do sensor para um estado fixo (cor azul, ciclos=0) usando um comando pre-gravado no firmware, sem parametros — mais rapido e mais confiavel na transmissao do que k044_fp_led_config(), mas sempre na cor azul e sem possibilidade de transicao de cor. Use k044_fp_led_config() quando precisar de outra cor ou de uma transicao entre cores.

Retorno: K044_OK ou erro (cada uma)

k044_fp_read_nfpage(k044_fp_nfpage_t *page)

Le a pagina de informacoes do modulo (numero de registros, tamanho do banco, numero de serie, versao de firmware, fabricante, nome do sensor, etc.).

page: ponteiro de saida (struct k044_fp_nfpage_t) que recebe os dados lidos; pode ser NULL se voce so quiser confirmar que o modulo responde, sem se importar com o conteudo.

Retorno: K044_OK ou erro

k044_fp_sleep(void)

Coloca o modulo em modo de baixo consumo. Qualquer rotina k044_fp_* chamada depois disso desperta o modulo automaticamente.

Retorno: K044_OK ou erro

k044_fp_led_config(uint8_t func, uint8_t start_color, uint8_t end_color, uint8_t cycle_times)

Controla o LED do sensor de digital, com controle total sobre modo, cor(es) e numero de ciclos (ao contrario das rotinas k044_fp_led_*_fast(), que sao pre-fixadas em azul).

func: modo do LED — uma das constantes K044_FP_LED_* (ver tabela no final desta secao).

start_color: cor inicial — uma das constantes K044_FP_LED_COLOR_* (ver tabela no final desta secao).

end_color: cor final — use o mesmo valor de start_color para uma cor solida, ou um valor diferente para fazer uma transicao entre as duas cores.

cycle_times: numero de ciclos do efeito (0 = infinito, repete ate o proximo comando de LED).

Retorno: K044_OK, ou K044_ERR_FP se o sensor rejeitar o comando

k044_fp_auto_identify(uint8_t score_level, int16_t id_number, uint16_t *found_id, uint16_t *score)

Identificacao automatica: captura a imagem, gera o template e busca no banco, tudo em um unico comando (equivalente a get_image+gen_char+search encadeados internamente).

score_level: nivel de exigencia do match, de 1 (mais permissivo) a 9 (mais rigoroso) — recomendado 5.

id_number: -1 para buscar em todo o banco; ou um ID especifico (0-999) para verificar 1:1 apenas contra aquele registro.

found_id: ponteiro de saida — recebe o ID encontrado.

score: ponteiro de saida — recebe a pontuacao de confianca do match.

Retorno: K044_OK se identificado, K044_ERR_FP se nao

k044_fp_auto_enroll(uint16_t id, uint8_t entries)

Cadastro automatico multi-etapa: o proprio modulo guia o processo internamente (pedir o dedo, capturar, pedir de novo, mesclar e gravar), notificando o progresso via callback (veja k044_fp_set_callback()).

id: identificador onde o cadastro sera gravado (1-999).

entries: numero de leituras/amostras do dedo a realizar (1-3, recomendado 3, para maior precisao).

Prefira k044_fp_enroll() como alternativa mais previsivel — ela reimplementa o mesmo fluxo chamando as rotinas basicas diretamente, sem depender do AutoEnroll interno do firmware.

Retorno: K044_OK ou erro

k044_fp_set_callback(k044_fp_callback_t cb, void *userdata)

Registra uma funcao de callback para acompanhar, em tempo real, o progresso de uma operacao demorada do fingerprint (cadastro ou busca) — util para dar feedback ao usuario (ex.: "coloque o dedo", "capturado, retire o dedo", "gravando...") sem precisar ficar consultando o estado manualmente.

cb: ponteiro para uma funcao com a assinatura

void (*)(int status, int step, void *userdata). NULL desregistra o callback atual. Os dois primeiros parametros recebidos por essa funcao, quando chamada pela biblioteca, significam:

status — 0x00 quando o passo correspondente teve sucesso; qualquer outro valor e' um codigo de confirmacao do sensor (o mesmo tipo de codigo devolvido por k044_fp_last_confirmation_code()) — passe esse valor para k044_fp_confirmation_str() para obter uma descricao textual do que aconteceu.

step — o passo atual da operacao em andamento. O significado depende de qual funcao disparou o callback:

* k044_fp_enroll() e k044_fp_search_retry() (rotinas de alto nivel desta biblioteca): usam as constantes K044_FP_STEP_WAIT_FINGER (1), K044_FP_STEP_CAPTURED (2), K044_FP_STEP_REMOVE_FINGER (3), K044_FP_STEP_MERGING (4) e K044_FP_STEP_STORING (5).

* k044_fp_auto_enroll() (processo interno do proprio firmware): usa uma numeracao propria do modulo (0=captura de imagem, 1=extracao de caracteristica, 2=gravacao, etc.) — nao ligada as constantes K044_FP_STEP_* acima.

userdata: ponteiro opaco definido por voce, repassado sem alteracao a cada chamada do callback — use para identificar contexto (ex.: qual tela ou objeto da sua aplicacao disparou a operacao).

Retorno: void

k044_fp_enroll(uint16_t id, int finger_timeout_ms)

Cadastro manual em 3 leituras (encadeia GetImage+GenChar+RegModel+Store internamente), alternativa mais previsivel ao AutoEnroll do firmware (k044_fp_auto_enroll()). Reporta o progresso via callback, se registrado (veja k044_fp_set_callback()).

id: identificador onde o cadastro sera gravado (1-999).

finger_timeout_ms: tempo maximo de espera pelo dedo, em milissegundos, em cada uma das 3 capturas — se o tempo esgotar sem detectar o dedo, a funcao retorna K044_ERR_FP_TIMEOUT.

Retorno: K044_OK, ou K044_ERR_FP/K044_ERR_FP_TIMEOUT

k044_fp_search_retry(uint8_t buffer_id, uint16_t start_page, uint16_t page_num, int finger_timeout_ms, int max_attempts, uint16_t *found_id)

Identificacao com repeticao automatica: captura e busca no banco, e tenta de novo em caso de falha, ate max_attempts tentativas.

buffer_id: qual CharBuffer usar internamente para gerar o template de cada tentativa — 1 ou 2.

start_page: posicao inicial do banco a partir de onde a busca comeca.

page_num: quantas posicoes, a partir de start_page, devem ser varridas em cada tentativa de busca.

finger_timeout_ms: tempo maximo de espera pelo dedo, em milissegundos, em cada tentativa.

max_attempts: numero maximo de tentativas de captura+busca antes de desistir.

found_id: ponteiro de saida — recebe o ID encontrado, quando ha sucesso.

Retorno: K044_OK se encontrou, K044_ERR_FP/K044_ERR_FP_TIMEOUT senao

k044_fp_confirmation_str(int code)

Traduz um codigo de confirmacao do modulo em uma descricao textual legivel.

code: o codigo a traduzir (0-53) — normalmente o valor recebido no parametro status de um callback (k044_fp_set_callback()) ou devolvido pela propria k044_fp_last_confirmation_code().

Retorno: const char (string estatica, nao precisa ser liberada)*

k044_fp_last_confirmation_code(void)

Retorna o ultimo codigo de confirmacao recebido de qualquer chamada

k044_fp_* — use para detalhar uma falha reportada como K044_ERR_FP, passando o valor retornado para k044_fp_confirmation_str().

Retorno: int (codigo, ou 0xFF se a resposta nao pode ser interpretada)

k044_fp_raw_test(const uint8_t *data, uint16_t data_len, uint8_t *resp, uint16_t *resp_len, uint16_t max_resp)

Diagnostico de baixo nivel: NAO usar para comunicacao normal com o modulo. Usado apenas para testar o sensor isoladamente, enviando um pacote bruto e observando a resposta crua.

data: buffer com os bytes a enviar diretamente ao sensor.

data_len: quantidade de bytes em data a enviar.

resp: buffer de saida onde a resposta do sensor e' copiada.

resp_len: ponteiro de saida — recebe a quantidade real de bytes recebidos na resposta.

max_resp: capacidade do buffer resp, em bytes (limite maximo que pode ser copiado para ele).

Retorno: K044_OK se o comando foi aceito e a transacao completou

Constantes de k044_fp_led_config()

Valores possíveis para os parâmetros func, start_color e end_color de k044_fp_led_config() (seção anterior).

CONSTANTE (FUNC)	VALOR	EFEITO
K044_FP_LED_BREATHING	1	Respiração (fade in/out contínuo)
K044_FP_LED_FLASHING	2	Piscando
K044_FP_LED_ALWAYS_ON	3	Sempre aceso
K044_FP_LED_ALWAYS_OFF	4	Sempre apagado
K044_FP_LED_GRADUAL_ON	5	Acende gradualmente
K044_FP_LED_GRADUAL_OFF	6	Apaga gradualmente

Cores para start_color/end_color — bitmask de 3 bits (bit0=azul, bit1=verde, bit2=vermelho):

CONSTANTE (COR)	VALOR	BITS (A/V/M)
K044_FP_LED_COLOR_BLUE	1	001 — azul
K044_FP_LED_COLOR_GREEN	2	010 — verde
K044_FP_LED_COLOR_CYAN	3	011 — azul + verde
K044_FP_LED_COLOR_RED	4	100 — vermelho
K044_FP_LED_COLOR_MAGENTA	5	101 — azul + vermelho
K044_FP_LED_COLOR_YELLOW	6	110 — verde + vermelho
K044_FP_LED_COLOR_WHITE	7	111 — todas



Soluções de tecnologia personalizadas para automação comercial, bancária e industrial de alta complexidade.

Suporte e Contato

WHATSAPP

(11) 4051-2781

E-MAIL

vendas@avanttectecnologia.com.br

HORÁRIO

Seg. à Sex. 8:00–17:30

Avanttec Tecnologia

CNPJ: 30.472.190/0001-00

Rua Puruboras, 61 – Conceição - Diadema/SP

CEP: 09990-340